

Latency Prediction for LLM Inference on NPU Systems

Juhyun Park¹, Seungwoo Jeong¹, Jingyu Lee², and Kyungyong Lee¹

¹Department of Data Science, Hanyang University, Seoul, Republic of Korea

²Department of Artificial Intelligence, Hanyang University, Seoul, Republic of Korea
{anchovypark, swjeong25, lj8540009, kyungyong}@hanyang.ac.kr

Abstract

Deploying Large Language Models (LLMs) requires exploring a large configuration space spanning parallelization strategies, batching techniques, and scheduling policies. Exhaustive measurement across this space is impractical, making latency prediction essential for system optimization. While NPUs have emerged as accelerators designed for LLM inference, no prediction methodology has been established for them. Specifically, applying prior work to LLM inference latency prediction on NPUs faces three challenges: undisclosed microarchitecture of commercial NPUs, unpredictable compiler optimizations, and latency non-linearity induced by bucketing.

We present LENS, a latency estimator that predicts NPU inference latency without information on the microarchitecture or compiler, and captures the non-linear latency induced by bucketing. LENS profiles each bucket with two end-to-end (E2E) measurements and composes the results to predict latency for arbitrary input-output length combinations. We validate LENS across NPUs from multiple vendors, several LLMs, and diverse workloads, achieving a mean prediction error of 2.15%. We further compare LENS against two methodologically related baselines, confirming the validity of its approach.

1. Introduction

The Transformer architecture [34] has driven rapid advances in Large Language Models (LLMs), and GPUs have served as the primary accelerator for this progress. LLM inference, however, exposes a structural mismatch with GPU architecture. The decode phase reads the entire accumulated KV cache from memory at every step, making memory bandwidth the dominant bottleneck [30, 41]. In addition, matmul and non-matmul kernels execute sequentially on the same SM, limiting hardware utilization [30, 39]. These limitations have led to the emergence of NPUs, domain-specific accelerators designed for LLM inference [20]. NPUs currently serve as production infrastructure for LLM deployment, including Anthropic’s Claude [5, 6] and Google’s Gemini [14].

Operating NPU-based serving systems requires exploring a vast configuration space spanning accelerator selection, batch size, and parallelization strategies. On NPUs, this exploration is further amplified by static compilation, which requires a separate compilation for each configuration.

Despite this need, no prediction methodology has been established for NPUs.

To enable LLM inference latency prediction on NPUs, we identify three challenges that block the direct application of prior work. First, the microarchitectural details of commercial NPUs are not disclosed at the level required for cycle-accurate simulation. Simulation-based NPU studies [15, 36] require detailed microarchitectural information as input, including systolic array dimensions, memory hierarchy, and Network-on-Chip (NoC) topology, yet recent commercial NPUs do not sufficiently disclose such information.

Second, the compilers of commercial NPUs apply graph optimizations in unpredictable ways. The physical separation of heterogeneous engines drives NPUs to form kernel fusion and optimization patterns that differ qualitatively from those of GPUs. When GPU-based methodologies [2, 7, 22, 38], which sum latencies measured at pre-defined units, are directly applied to NPUs, error rates of up to 493% are observed.

Third, the static compilation of NPUs renders latency a discontinuous function of sequence length. Since each input shape requires a separate compilation, NPU-based serving systems adopt a bucketing strategy that pads inputs to pre-defined lengths, causing latency to jump discretely at bucket boundaries. This step-function distribution invalidates the assumption of existing prediction methodologies, which interpolate continuous functions between measurement points.

To address these challenges, we leverage two observations: the end-to-end (E2E) latency of a compiled binary is externally observable, and all inputs within a bucket share the same compiled binary. Building on these observations, we present LENS (Latency Estimator for NPU Systems), which predicts LLM inference latency for arbitrary input-output length combinations from only two E2E measurements per bucket.

- Characterization of three challenges that obstruct LLM inference latency prediction on commercial NPUs.
- Reinterpretation of the bucketing structure inherent in NPU serving systems as a natural unit of measurement for prediction, together with the design of LENS, a prediction tool that treats NPUs as black boxes and operates solely on per-bucket E2E profiling.
- Validation of LENS across NPUs from multiple vendors, several LLMs, and diverse workloads, achieving a mean

prediction error of 2.15%, and comparison against two methodologically related baselines.

- Demonstration through two case studies that NPU configuration optima cannot be reasoned about without direct measurement, with LENS supporting this search through per-bucket E2E profiling.

2. NPU for LLM Serving Systems

NPUs are hardware accelerators designed for LLM inference [19, 35, 37]. This section examines the architectural design of NPUs for this workload, and the research direction needed for their effective use.

2.1. LLM Inference on GPUs

Workloads dominated by large-scale matrix multiplication, such as LLM training, are executed efficiently on the massively parallel compute cores of GPUs [26, 28]. Modern GPUs further accelerate this operation through dedicated Tensor Cores. This alignment has driven the success of GPU-based LLM training.

LLM inference, however, splits into two phases with distinct computational characteristics due to the Key/Value (KV) cache. Prefill processes the entire input prompt in parallel, and matrix multiplication still dominates this phase. Decode then generates output tokens one at a time autoregressively. By reusing cached KV tensors from previous iterations, each decode iteration processes solely the newly generated token, and matrix-vector multiplication, which has low arithmetic intensity, dominates this phase.

These characteristics of LLM inference expose two structural mismatches with GPU architecture. First, the low arithmetic intensity renders decode memory-bound, and the parallel compute cores of GPUs remain underutilized [30, 41]. Second, Tensor Cores and CUDA Cores are co-located on the same SM, which forces matmul and non-matmul kernels to execute serially. With the runtime share of matrix multiplication reduced relative to prefill, non-matmul kernels (e.g., element-wise operations, activations, normalizations) account for a substantial fraction of decode latency [3, 30]. During decode, matrix-vector multiplication occupies the Tensor Cores in a memory-bound state, and the colocated CUDA Cores remain idle for the duration of the kernel. Non-matmul kernels cannot be dispatched to these idle resources and must await completion of the matmul kernel. The GPU therefore fails to reach peak throughput during inference [11].

2.2. NPUs for LLM inference

The mismatch between GPU architectures and LLM inference has motivated Neural Processing Units (NPUs) [20]. NPUs address both mismatches — compute underutilization under low arithmetic intensity, and forced serialization of heterogeneous kernel execution — through architectural choices specialized for LLM inference [19, 35, 37]. First, NPUs adopt dataflow architectures that maximize on-chip data reuse at the hardware level. Figure 1 illustrates the architecture of a systolic array under a weight-stationary

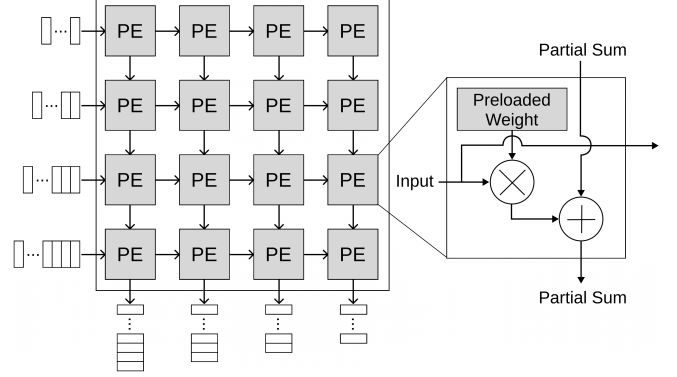


Figure 1: Structure of a systolic array.

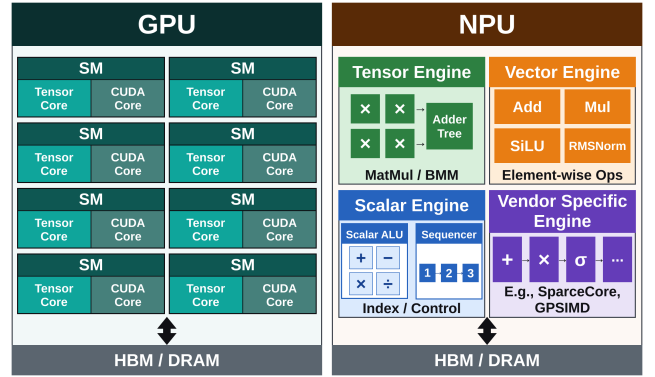


Figure 2: A representative architecture of an NPU.

dataflow. Data streams sequentially through processing elements (PEs), and the operands (i.e., weights) resident in each PE are reused across multiple computations, minimizing off-chip memory accesses. This hardware-level data reuse pattern effectively alleviates the memory bandwidth bottleneck of the memory-bound decode phase.

Second, GPUs execute all operations on shared SMs regardless of operation type, whereas NPUs employ physically separate engines for each operation type. Figure 2 contrasts the architectures of GPUs and NPUs. An NPU is typically partitioned into a Tensor Engine, which handles matrix multiplications; a Vector Engine, which handles element-wise operations, softmax, and normalization; and a Scalar Engine, which handles index computation and loop counters. Each NPU may further include vendor-specific engines (e.g., Google TPU’s SparseCore, AWS NeuronCore’s GPSIMD). These engines are physically separated, and the output of one engine can be pipelined directly into the input of another through on-chip pathways without traversing HBM. The separate-engine architecture enables heterogeneous operations to execute concurrently, thereby minimizing the device underutilization that arises from serial execution.

2.3. LLM Inference Latency Prediction

The deployment of LLM serving systems must satisfy a Service Level Objective (SLO) that bounds the response

TABLE 1: Prior work on LLM inference latency prediction in GPU environments. Categorized by the granularity of decomposition.

Decomposition Unit	Representative Work	Prediction Approach
GPU kernel	NeuSight [22] Habitat [38]	Estimate per-kernel execution time and sum
Logical unit (operator / layer)	Vidur [2] MaverIQ [25]	Extrapolate per-unit profiling results and sum

latency for user requests, while maximizing throughput under a limited budget. Meeting these goals involves multiple design decisions, including the selection of accelerator, batch size, and parallelization strategies such as Tensor Parallelism (TP) and Pipeline Parallelism (PP), each of which entails trade-offs among latency, throughput, and cost. Identifying the optimal configuration within this multidimensional trade-off space is therefore essential.

However, exploring this configuration space is prohibitively expensive in practice, as the number of candidate configurations explodes combinatorially across the design dimensions. Accurately characterizing the performance of each candidate configuration requires deploying and executing the model on the target platform, rendering exhaustive measurement infeasible. Vidur [2] reports that exploring the optimal deployment configurations of LLaMA2-70B across two GPUs (A100 and H100) and three workloads requires 42,000 GPU-hours, highlighting the necessity of latency prediction.

Despite this necessity, predicting LLM inference latency is non-trivial. The decode phase incurs computation and memory access volumes that grow non-linearly with input and output lengths, since the KV cache accumulates at each step. These dynamics combine with batch size, sequence length, hardware utilization, and bandwidth contention at runtime. Analytical models that treat latency as a simple function of input variables therefore incur substantial errors. NeuSight [22] demonstrates that applying the linear regression-based prediction approach [24] from conventional DNN settings to LLM inference yields an error rate of approximately 61.2%.

To address these challenges, various prediction methodologies have been developed for GPU environments, as summarized in Table 1. While these approaches differ in their specifics, they share a common formulation that decomposes LLM inference into pre-defined units and estimates E2E latency by summing the per-unit latencies. In contrast, no such prediction methodology has been established for NPUs, despite the higher exploration cost imposed by their static compilation characteristics. We address this gap with LENS, a latency prediction methodology for NPUs.

3. Challenges of Latency Prediction in NPUs

Latency prediction is a critical capability for operating NPU-based serving systems, yet three challenges complicate it. These challenges preclude both the adaptation of existing

TABLE 2: Microarchitectural disclosure at simulation grade across commercial NPUs. Each row is an input category required by prior NPU simulators [16, 32, 36].

Input Category	Google TPU			AWS
	v4 [19]	v5e [12]	v6e [13]	Inferentia2 [4]
Systolic / MAC Array	✓	✓	✓	✓
Vector / Scalar Unit	×	✓	×	✓
On-chip Memory	×	×	×	✓
Off-chip Memory (HBM)	×	×	×	×
Interconnect	×	×	×	×
Clock Frequency	✓	×	×	×
ISA / Prog. Model	×	×	×	✓

NPU studies to the prediction task and the application of prior GPU prediction methodologies to NPUs.

3.1. Undisclosed Microarchitecture of NPUs

Most prior NPU research has focused on simulation tools for design space exploration (DSE) of new NPU hardware [15, 29, 32, 36]. As a representative example, PyTorchSim [36] converts PyTorch-level model definitions to MLIR via Aten IR and performs cycle-level simulation on a systolic array-based virtual NPU, achieving a mean absolute error of 11.5% when validated against the TPU v3 architecture. Similarly, ONNXim [15] performs cycle-level simulation on ONNX graphs and enables DSE of multi-core NPUs at simulation speeds 384× faster than prior accelerator simulators.

However, such simulation-based approaches face structural limitations on NPUs, whose microarchitectural details are not fully disclosed. As shown in Table 2, the four NPUs commonly disclose only the dimensions and counts of their systolic arrays. None reveals the parameters essential for cycle-accurate simulation, such as cycle-level HBM timings or NoC routing policies.

To quantify the impact of this limitation, we conduct cycle-level simulation of Inferentia2 using PyTorchSim, with undisclosed fields filled by estimated values. While standalone MatMul is reproduced reasonably well with a 7.6% error, element-wise and reduction operations exhibit errors as large as 232–323%. These errors stem from microarchitectural opacity [23, 36]. The authors of these simulators note that undisclosed microarchitectural parameters preclude exact cycle-count matching with the target hardware, and attribute the residual errors in their own validations to the same root cause.

3.2. Unpredictable Compiler Optimizations

NPUs rely on vendor-provided compilers that lower model graphs to the underlying microarchitecture, applying graph optimizations such as kernel fusion, memory allocation, and execution scheduling. Compiler optimization is not unique to NPUs, but its scope on NPUs differs sharply from that on GPUs. On GPUs, automatic optimization rarely goes beyond fusing adjacent operations into a single kernel to reduce HBM round-trips [40], unless developers introduce hand-written fusion kernels such as FlashAttention [9]. On

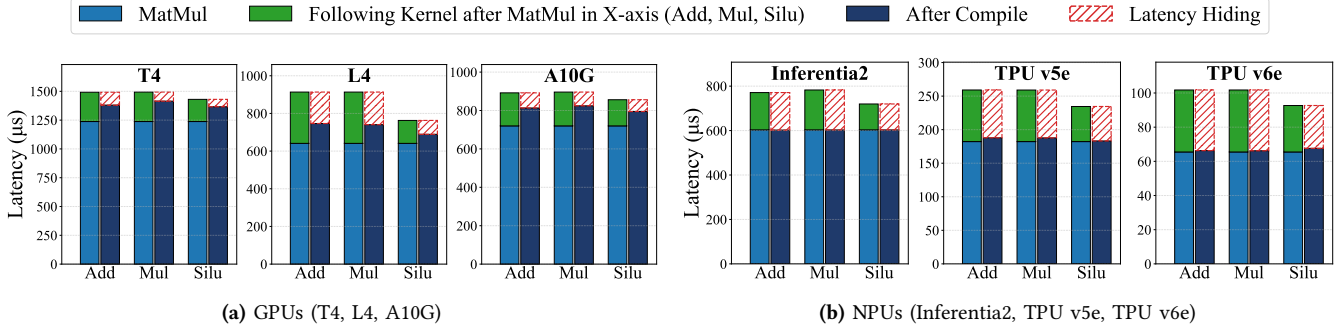


Figure 3: Kernel fusion difference between NPUs and GPUs. Blue bars show MatMul latency, green bars show the following kernel’s latency (operation type on the x-axis). The stacked bars sum the individually measured latencies, while “After Compile” bars show the latency of the two operations executed back-to-back. The red hatched boxes indicate the latency reduction from compiler optimization.

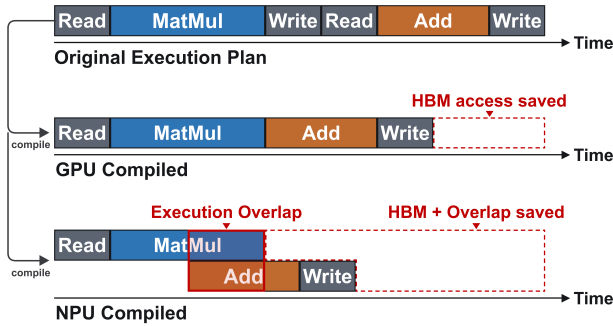


Figure 4: Compilation difference between GPUs and NPUs. On GPUs, compiler optimization saves only the HBM access time between two kernels. On NPUs, the compiler can schedule the two operations to execute concurrently on separate engines.

NPUs, by contrast, matrix, vector, and scalar operations run on distinct engines, so the compiler can schedule adjacent operations to execute concurrently. As Figure 4 shows, such concurrent execution can hide one kernel’s latency within another’s [18–20].

We measure this difference with a controlled experiment that chains a MatMul on the Tensor Engine with a subsequent operation (Add, Mul, or SiLU) on a separate engine through a data dependency. For each setup, we measure the standalone latencies of the two operations and the latency of their chained execution. As Figure 3 shows, on NPUs the latency of the kernel following MatMul is nearly hidden and the total latency stays close to that of MatMul alone, whereas on GPUs the second kernel’s latency is only partially reduced and no comparable hiding occurs. If even two operations produce such divergent behavior, predicting compiler optimizations across the hundreds of operations in an LLM inference graph is non-trivial.

The unpredictability of NPU compiler optimizations prevents the transfer of GPU-based prediction methodologies [2, 22, 25, 38] to NPUs. These GPU-based approaches commonly assume that individual kernels execute sequentially on the device, an assumption empirically supported on GPUs [42]. On NPUs, however, the compiler optimizations

TABLE 3: Prediction errors of GPU-based methodologies when applied to NPUs. Each column corresponds to a GPU-based latency prediction methodology, and each row corresponds to an input/output length configuration.

Input/Output	Kernel-sum Error (%) [22, 38]	Operator-sum Error (%) [2]	Layer-sum Error (%) [25]
512/512	+80.9	−63.58	+22.02
1024/1024	+493.3	−60.64	−43.00
2048/2048	+446.0	−57.37	−11.35

applied during per-unit profiling do not necessarily match those applied during E2E execution. This mismatch arises regardless of whether the decomposition unit is a kernel or a logical operator. Table 3 reports the prediction errors of these methodologies when applied to NPUs.

3.3. Non-linear Latency Induced by Bucketing

NPUs support only static compilation, requiring a separate compilation for every input shape. However, LLM inference involves requests with varying input and output lengths, making per-length compilation impractical. To address this, NPU-based LLM serving systems adopt a *bucketing* strategy: the compiler emits binaries only for a set of predefined lengths (e.g., 256, 512, 1024, 2048, ...), and incoming inputs are padded up to the nearest bucket size. Each bucket corresponds to a separately compiled binary, and inputs assigned to the same bucket are executed by the same binary. Figure 5 plots the TTFT against input length, with each point measured at a different input length within the same bucket. Latency follows a step-function pattern, remaining constant within each bucket and jumping sharply at bucket boundaries. The overall LLM inference latency on NPUs is therefore governed by the sequence of buckets traversed during inference.

This bucketing structure carries two important implications for LLM inference latency prediction on NPUs. First, prediction approaches that assume a continuous latency function and interpolate between measurement points break down under this behavior. For example, MaverIQ [25]

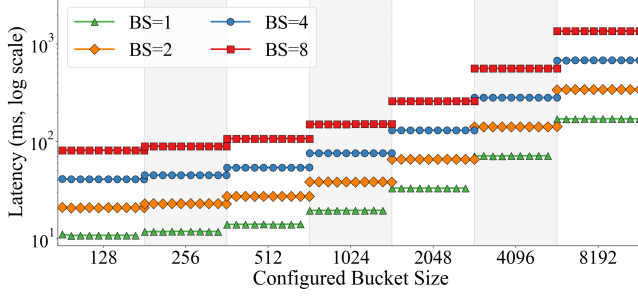


Figure 5: Step latency induced by the bucketing effect on NPUs. TTFT measured per bucket on TPU v6e with Llama 3.1 8B deployed in a TP4 configuration, while varying the batch size across 1, 2, 4, 8. Each point corresponds to a different input length assigned to the bucket.

interpolates LLM inference latency from a small number of measurement points, but this continuity assumption fails at bucket boundaries. As shown in Figure 5, a single-token increase across the 4096-to-8192 boundary multiplies TTFT by approximately 2.4× (e.g., from 71 ms to 171 ms at batch size 1), well outside the range that interpolation-based methods can absorb.

Second, profiling through actual execution at each measurement point becomes substantially more expensive on NPUs than on GPUs. Prior work [2, 7, 25] reports profiling costs of several to tens of hours per model in GPU environments, and NPUs add a separate compilation step whenever the input shape changes. The official JAX documentation [33] similarly notes that the cumulative compilation cost can dominate the actual profiling time as the number of measurement points grows. A practical prediction system for NPUs therefore must explicitly exploit the bucketing structure to keep the number of measurement points tractable.

Consequently, the challenges identified in this section imply three requirements for LLM latency prediction methodology on NPU systems:

- R1.** Prediction without information on the NPU microarchitecture or compiler behavior.
- R2.** Reflection of bucketing-induced latency non-linearity.
- R3.** Efficient profiling despite static compilation cost.

4. LENS: Latency Estimator for NPU Systems

To satisfy the requirements for prediction methodology on NPUs, we build LENS on two key observations. First, the E2E latency of a compiled binary is measurable even when the microarchitecture and compiler are treated as black boxes, making it a viable unit of prediction (R1). Second, lengths within a bucket share the same compiled binary, so each bucket can be characterized from a few measurements without additional compilation (R2, R3).

Building on these observations, we propose LENS, a latency estimator that predicts the E2E latency of LLM inference on NPUs. LENS profiles each bucket with only a few measurements and composes the resulting per-bucket

values to predict the latency of arbitrary input-output length combinations.

4.1. Prediction Model

LLM inference consists of a prefill phase and a decode phase. Since the latency of each phase is determined by a different bucket, LENS models the E2E latency as the sum of the prefill and decode latencies.

Prefill. In the prefill phase, the entire input sequence is processed at once, and its latency corresponds to the TTFT. On NPUs, the TTFT is determined not by the actual input length but by the bucket to which the input is assigned. We define $TTFT_b$ as the prefill latency of bucket b .

Let B denote the set of bucket sizes configured on the NPU. Given an input length l^{in} , we define the function that assigns it to a bucket as:

$$\text{bucket}(l^{\text{in}}) = \min\{b \in B \mid l^{\text{in}} \leq b\} \quad (1)$$

The TTFT of a single request is then:

$$T_{\text{prefill}} = TTFT_{\text{bucket}(l^{\text{in}})} \quad (2)$$

Decode. In the decode phase, tokens are generated one step at a time, and the per-step latency corresponds to the TBT (Time Between Tokens). While the prefill bucket is determined by the input length l^{in} , the decode bucket at each step is determined by the size of the KV cache at that step. Let c_t denote the KV cache length at generation step t . Since c_t equals the input length plus the number of tokens generated so far, it can be expressed as:

$$c_t = l^{\text{in}} + t + 1, \quad 0 \leq t < l^{\text{out}} \quad (3)$$

We define TBT_b as the per-step decode latency of bucket b . The bucket used at step t is $\text{bucket}(c_t)$, and the latency of that step is $TBT_{\text{bucket}(c_t)}$. If the KV cache stays within a single bucket b throughout decoding, the decode latency for l^{out} tokens is:

$$T_{\text{decode}} = l^{\text{out}} \cdot TBT_b \quad (4)$$

In practice, c_t grows monotonically as decode proceeds, so c_t eventually exceeds the upper bound of the current bucket and the system switches to the binary of the next bucket. The decode process is therefore divided into multiple segments, each with a different TBT_b . Let K be the total number of segments, b_j the bucket used in segment j , and l_j^{out} the number of tokens generated within that bucket. The decode latency is then:

$$T_{\text{decode}} = l_1^{\text{out}} \cdot TBT_{b_1} + l_2^{\text{out}} \cdot TBT_{b_2} + \dots = \sum_{j=1}^K l_j^{\text{out}} \cdot TBT_{b_j} \quad (5)$$

Combining prefill and decode, the LENS E2E latency prediction for a single request is:

$$T_{\text{E2E}} = T_{\text{prefill}} + T_{\text{decode}} = TTFT_{\text{bucket}(l^{\text{in}})} + \sum_{j=1}^K l_j^{\text{out}} \cdot TBT_{b_j} \quad (6)$$

4.2. Extension to Batched Inference

In practice, LLM serving systems batch multiple requests together to improve throughput. The single-request formulation introduced above therefore needs to extend to the batched setting.

Batched Prefill. NPU serving frameworks process each request in a batch independently in its own bucket. Therefore, the batch prefill latency for a batch of N requests $\mathcal{R} = \{r_1, r_2, \dots, r_N\}$ is the sum of the per-request prefill costs:

$$T_{\text{prefill}}(\mathcal{R}) = \sum_{i=1}^N \text{TTFT}_{\text{bucket}(l_i^{\text{in}})} \quad (7)$$

Batched Decode. Unlike prefill, where each request uses its own bucket, decode executes the entire batch with a single fixed shape. The TBT at each step is therefore determined by the bucket containing c_t^{max} , the maximum KV position referenced by the batch:

$$c_t^{\text{max}} = \max_{i \in \mathcal{R}} c_t^{(i)} \quad (8)$$

Furthermore, requests with shorter outputs are padded and executed alongside the others until the longest output completes, so the range of t is determined by the longest output length in the batch:

$$0 \leq t < \max_{i \in \mathcal{R}} l_i^{\text{out}} \quad (9)$$

In the single-request case, we denoted the number of tokens generated within bucket b_j as l_j^{out} . In the batched setting, however, the timing of bucket transitions is determined not by individual output lengths but by the trajectory of c_t^{max} across the entire batch. Let n_j denote the number of steps that c_t^{max} stays within bucket b_j . The batched decode latency is then:

$$T_{\text{decode}}(\mathcal{R}) = \sum_{j=1}^K n_j \cdot \text{TBT}_{b_j} \quad (10)$$

Combining prefill and decode, the LENS batched E2E latency prediction is:

$$\begin{aligned} T_{\text{E2E}}(\mathcal{R}) &= T_{\text{prefill}}(\mathcal{R}) + T_{\text{decode}}(\mathcal{R}) \\ &= \sum_{i=1}^N \text{TTFT}_{\text{bucket}(l_i^{\text{in}})} + \sum_{j=1}^K n_j \cdot \text{TBT}_{b_j} \end{aligned} \quad (11)$$

Extension to Optimized Attention Kernels. Recent NPU serving frameworks introduce optimized attention kernels that allow each request to perform attention computation up to its actual KV length [17], mitigating padding inefficiency. The overall computation still follows bucketing-based static compilation. We approximate decode latency by assigning each request to the bucket matching its own KV length.

While Eq. 10 assigns a single bucket’s TBT to the entire batch, the batch still shares a single forward step under

these kernels. The per-request cost is therefore the bucket’s TBT divided by the batch size N .

$$\overline{\text{TBT}}_b = \frac{\text{TBT}_b}{N} \quad (12)$$

Each request then traverses buckets according to its own KV length, identical to the single-request model (Eq. 5). Applying Eq. 5 to each request with the normalized TBT, the batched decode latency becomes:

$$T_{\text{decode}}(\mathcal{R}) = \sum_{i=1}^N \sum_{j=1}^{K_i} l_j^{\text{out},(i)} \cdot \overline{\text{TBT}}_{b_j^{(i)}} \quad (13)$$

where K_i is the total number of buckets traversed by request i , $b_j^{(i)}$ is the j -th bucket of request i , and $l_j^{\text{out},(i)}$ is the number of tokens generated within that bucket.

4.3. Profiling Point

In the formulation of Eq. 11, the only measured quantities required by LENS are TTFT_b and TBT_b for each bucket b . Since the same binary executes for all input lengths within a bucket, both values can be obtained from only two E2E execution-time measurements per bucket. Specifically, for input length $l^{\text{in}} = b$, we measure the E2E latency with two different output lengths L_1 and L_2 , yielding:

$$T_{\text{E2E}}^{(1)} = \text{TTFT}_b + L_1 \cdot \text{TBT}_b, \quad T_{\text{E2E}}^{(2)} = \text{TTFT}_b + L_2 \cdot \text{TBT}_b \quad (14)$$

Solving this system yields:

$$\text{TBT}_b = \frac{T_{\text{E2E}}^{(2)} - T_{\text{E2E}}^{(1)}}{L_2 - L_1}, \quad \text{TTFT}_b = T_{\text{E2E}}^{(1)} - L_1 \cdot \text{TBT}_b \quad (15)$$

As a result, two measurements per bucket suffice to predict the latency for all input lengths within that bucket.

5. Evaluation

In this section, we validate that LENS achieves high prediction accuracy for arbitrary input-output length combinations from only a small number of profiling measurements. The evaluation covers four axes of variation. We test three models ranging from 1B to 14B parameters across four NPUs with different SDKs and architectures, sweep batch sizes from 1 to 32, and use four real-world datasets with diverse input-output length distributions.

5.1. Experimental Setup

NPU. LENS’s prediction approach is general and does not depend on any specific NPU or compiler stack. On AWS, we use Inferentia2 with the Neuron SDK, and on GCP, we use TPU v4, TPU v5e, and TPU v6e with JAX/XLA. These NPUs span a wide range of peak compute (190–918 BF16 TFLOPS) and HBM bandwidth (0.8–1.6 TB/s), while sharing the common characteristics of heterogeneous engine design and static compilation. All experiments use a 4-chip configuration, which corresponds to TP4 on TPUs, and to TP8 on Inferentia2 since it has 2 cores per chip.

TABLE 4: Dataset statistics before and after 8K-token filtering.

Dataset	Content	# queries	# prefill tokens			# decode tokens			P:D ratio
			mean	median	p90	mean	median	p90	
ShareGPT [1] (ShareGPT)	Real-world ChatGPT conversations	93K	167	23	403	327	296	666	1:2
	ShareGPT with max 8k total tokens	93K	167	23	403	327	296	666	1:2
CNN/DailyMail [27] (CNN)	News article summarization	312K	865	788	1,458	64	60	95	14:1
	CNN/DailyMail with input length > 1,024 tokens	92K	1,393	1,305	1,872	73	67	113	19:1
ArXiv Summarization [8] (ArXiv)	Summarization of arxiv papers	216K	8,605	6,879	16,078	355	202	411	24:1
	ArXiv-Summarization with max 8k total tokens	121K	4,822	4,894	7,229	207	175	326	23:1
Writing Prompts [10] (Writing Prompts)	Creative story generation from short prompts	303K	30	28	52	701	577	1,368	1:23
	Writing Prompts with max 8k total tokens	303K	30	28	52	701	577	1,368	1:23

Models. To ensure diversity in computational structure, we use three models from different families: Llama-3.2-1B, Mistral-7B-v0.3, and Qwen3-14B, covering parameter scales from 1B to 14B. We vary the batch size from 1 to 32, subject to each hardware’s memory capacity.

Datasets. To evaluate on diverse input-output workloads, we use four datasets. ShareGPT [1] consists of real-world ChatGPT conversations with short inputs but a long-tailed distribution. CNN/DailyMail [27] is a news summarization dataset, from which we extract only samples with input length above 1,024 tokens to cover the mid-length range. ArXiv Summarization [8] consists of academic papers with very long inputs (mean of 4,808 tokens). Writing Prompts [10] is a creative writing workload that generates long outputs from short inputs (mean output of 697 tokens), covering the long-output regime. The input-output length distributions of each dataset are summarized in Table 4. Samples exceeding 8,192 tokens in maximum sequence length are excluded to fit within each NPU’s HBM capacity.

Measurement Setup. We measure latency using the native serving framework of each NPU platform: NxDI on AWS Inferentia2 (Neuron SDK), and MaxText on GCP TPUs (JAX/XLA). Both frameworks execute LLM inference under the compiler’s full optimization pipeline, matching the bucket-level E2E latency that LENS predicts. We use a fixed bucket set $B = \{128, 256, 512, 1024, 2048, 4096, 8192\}$ across all experiments.

5.2. Prediction Accuracy in Diverse Settings

Figure 6 reports LENS’s E2E latency prediction errors across four NPU configurations (all with 4 chips), three models, and four datasets. The main evaluation covers 248 cases spanning four NPUs, three models, four datasets, and multiple batch sizes. LENS achieves a mean error of 2.15% (median 1.69%), with 89.9% of cases falling under 5% error.

Consistency across NPUs. LENS maintains low error rates across all four NPUs, despite their different compiler stacks and hardware architectures. The mean errors are 2.20% (max 8.07%) on Inferentia2-TP8, 2.65% (max 9.05%) on TPU v4-TP4, 2.35% (max 11.47%) on TPU v5e-TP4, and 1.46% (max 6.52%) on TPU v6e-TP4. These results indicate that bucket-based modeling remains effective across both compiler stacks (Neuron SDK and JAX/XLA) and hardware

architectures with different systolic array dimensions and memory hierarchies.

Consistency across model scales. LENS maintains stable accuracy from 1B (Llama-3.2-1B) to 14B (Qwen3-14B). The mean errors are 2.70% for Llama-3.2-1B, 1.61% for Mistral-7B, and 1.52% for Qwen3-14B, with errors decreasing as model size and batch size grow.

Consistency across workload characteristics. LENS maintains accurate predictions across datasets with diverse input-output length distributions. The mean errors are 1.21% on ArXiv (prefill-dominant) and 1.41% on Writing Prompts (decode-dominant), indicating that bucket-based modeling captures both extremes of the prefill-decode spectrum. ShareGPT and CNN/DailyMail show slightly higher mean errors of 2.49% and 3.49%, respectively. The shorter sequences in these datasets yield smaller absolute latencies, where measurement noise forms a larger fraction of the prediction error. All four datasets remain within 5% on average, confirming that dataset characteristics have little impact on prediction accuracy.

5.3. Comparison with Baselines

Direct baselines for NPU latency prediction are scarce, so we select two methodologically related works. LLM Serving Sim 2.0 [7] is a unified LLM serving simulator validated on NPUs. It sums per-operator latencies and applies an E2E correction step to mitigate the black box nature of compiler optimization, a strategy methodologically aligned with LENS.

MaverIQ [25] is a layer-level latency predictor for GPU-based LLM serving. It models latency as a linear function of token count, with a single TTFT constant and a single TBT slope across the entire dataset. LENS adopts a similar linear assumption but applies it per bucket, making MaverIQ a methodological baseline despite its GPU origin and global linearity.

Since LLM Serving Sim 2.0 is built on the vLLM-TPU codebase, we use vLLM-TPU as the serving framework for all methods.

Baseline-Results. LLM Serving Sim 2.0 and MaverIQ both achieve reasonable accuracy in certain regions but reveal large errors where their assumptions break.

LLM Serving Sim 2.0 errors amplify as batch size grows and prefill operations accumulate. On ArXiv under BS=32,

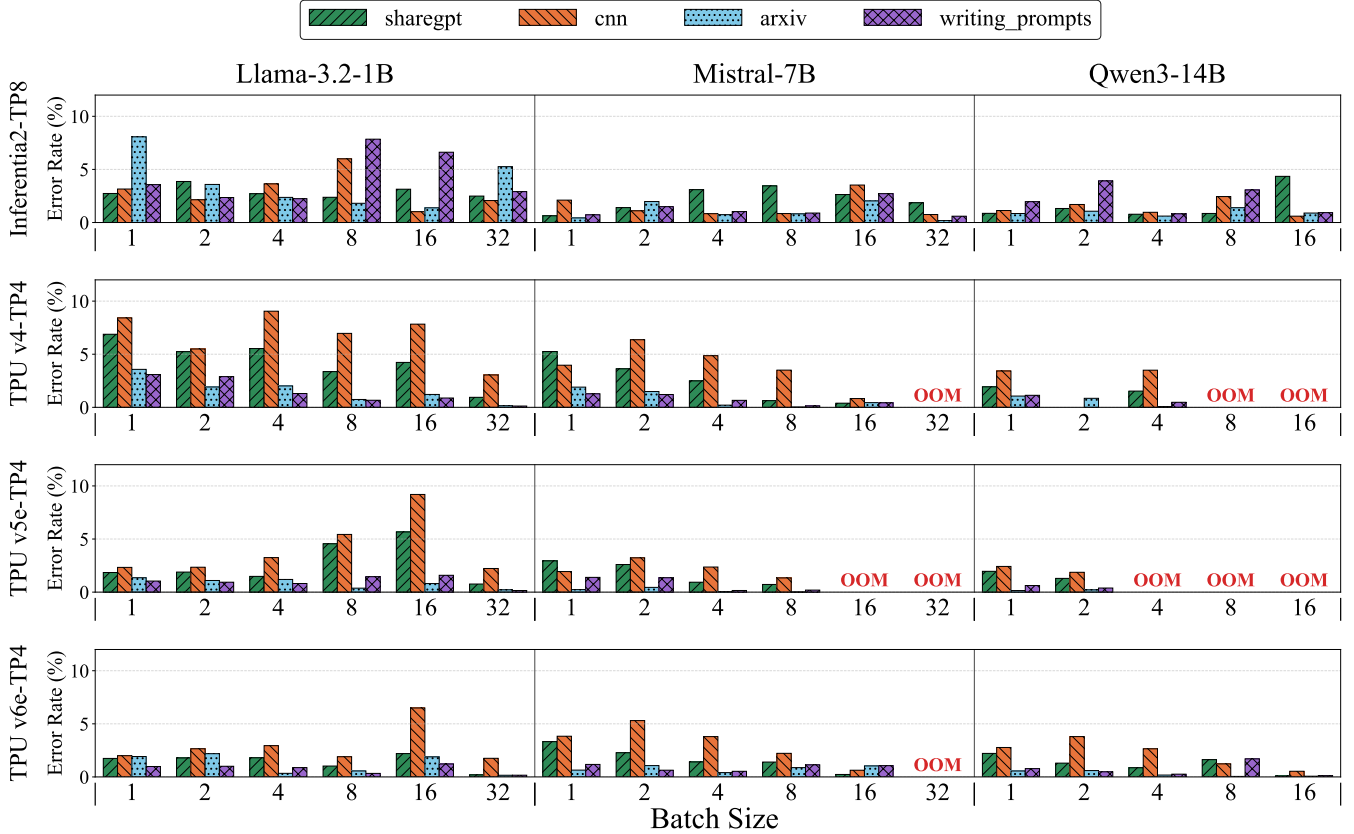


Figure 6: Accuracy evaluation of LENS on four NPUs. Each bar reports the prediction error against measured latency.

the error exceeds 200%. Despite the E2E latency correction step, per-operator summation fails to capture graph-level compiler optimizations.

MaverIQ assumes no bucketing. Fitting a single TTFT constant and TBT slope to an entire dataset, it incurs sharply growing errors on workloads where bucket transitions occur frequently during inference. On CNN, errors within the same dataset split between regions exceeding 80% and regions in the 40% range depending on batch size, and on ShareGPT, errors of 50–70% persist across all batch sizes. MaverIQ averages 33% Mean Absolute Percentage Error (MAPE) across all configurations.

LENS addresses both limitations. It uses E2E measurement as the unit of prediction, which captures compiler optimizations that per-operator summation misses, and it maintains a separate model per bucket, which captures the discontinuity at bucket transitions that a global linear fit cannot express. On the same configurations, LENS achieves an average MAPE of 5.3%.

Profiling Cost Comparison. LLM Serving Sim 2.0 demands the largest profiling budget. Per-operator profiling requires 5,000 measurement points, and each point requires a separate compilation. Total profiling time, including compilation, reaches 7 hours.

MaverIQ uses a single decoder layer as the profiling unit, which shortens execution time compared to E2E

measurement. However, each dataset requires a separate compilation graph, leading to 4 compilations per batch and 41 minutes of compilation needed across the experiment. This compilation cost dominates MaverIQ’s total profiling time of 45 minutes, leaving the layer-level profiling unit poorly matched to the actual cost structure.

LENS uses 14 profiling points per batch, but only 1 compilation per batch. The entire configuration requires 12 compilations in total. Compilation takes 10 minutes, and the total profiling time including execution is 18 minutes.

Reducing the profiling unit is therefore not sufficient on NPUs. Profiling cost is dominated by compilation, so the design must minimize the number of compilations. LENS uses more profiling points than MaverIQ but requires only 12 compilations against MaverIQ’s 48, matching the cost structure of NPU profiling.

5.4. Case Study: Where Intuitions Fail on NPUs

GPU intuitions about LLM serving do not carry over to NPUs. The compiler reshapes execution per configuration in ways that cannot be anticipated. NPU serving system operators therefore cannot reason about an optimum without measuring it. LENS serves as a key tool: it needs only a few profiling points, an advantage amplified on NPUs where compile time dominates profiling cost. The two case studies below illustrate this role.

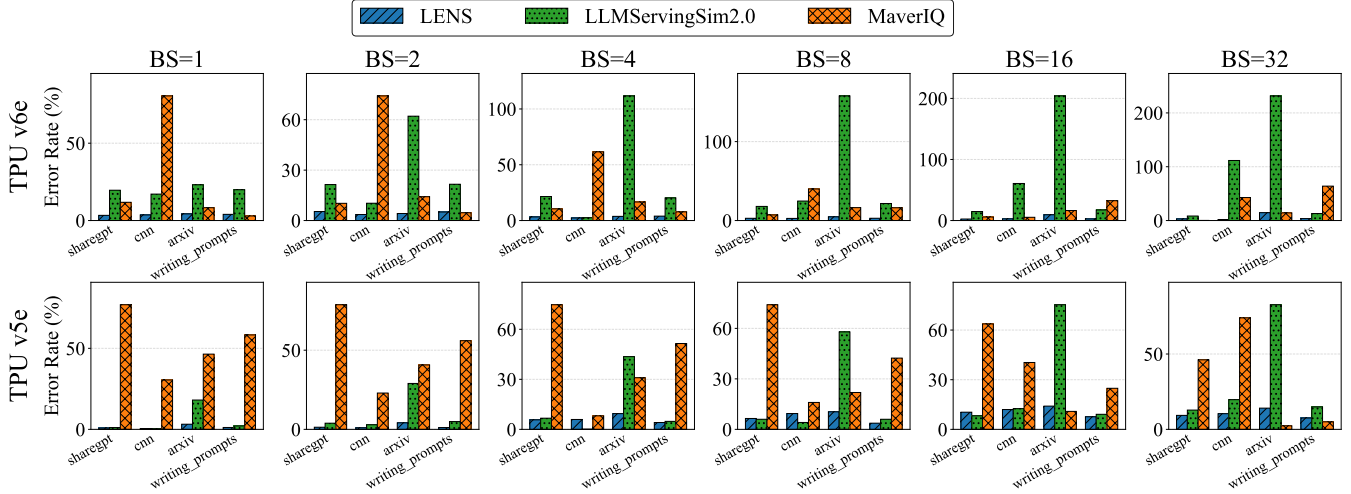


Figure 7: Comparison with Baseline

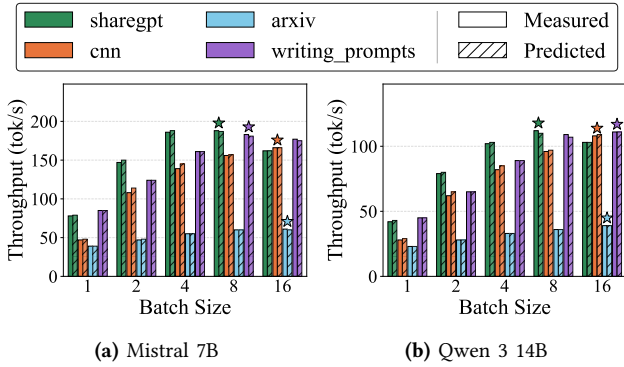


Figure 8: Measured and predicted throughput on TPU v6e across batch sizes for each dataset. Stars mark the batch size that achieves the highest throughput for each dataset.

5.4.1. Case Study 1: Larger Batches Do Not Always Maximize Throughput on NPUs. Larger batches on GPUs raise throughput at the cost of per-request latency [11, 21, 39]. Throughput climbs with batch size and then plateaus once the GPU saturates.

NPUs break this pattern. The NPU compiler is shape-sensitive, and the execution path it builds for a given input shape can be either highly optimized or far less efficient, depending on the shape itself [17]. The GPU intuition that larger batches always raise throughput therefore no longer holds on NPUs. Each batch size produces a distinct input shape, and the compiler may optimize one shape but not a neighboring one. A larger batch can therefore land on a slower path and deliver lower throughput than a smaller one.

Figure 8 illustrates this across two models. On TPU v6e with Mistral 7B, the throughput-maximizing batch size shifts with the dataset (Figure 8a): CNN and ArXiv peak at BS 16, while ShareGPT and Writing Prompts peak at BS 8.

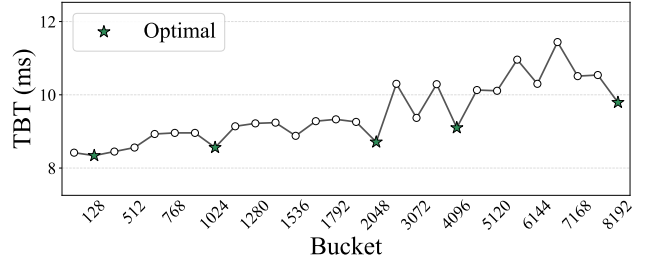


Figure 9: Measured TBT across buckets (Inferentia2, TP=2, Llama-3.2 1B, BS=1). Stars mark the buckets that lie on a compiler-optimized execution path.

The same pattern appears on Qwen3 14B (Figure 8b): most datasets peak at BS 16, but ShareGPT peaks at BS 8.

Each (LLM, NPU, dataset) combination thus requires its own search for the throughput-maximizing batch size. LENS makes this search practical, predicting these peaks accurately and locating them quickly, as Figure 8 confirms.

5.4.2. Case Study 2: Smaller Buckets Do Not Always Run Faster on NPUs. Bucket selection is a key decision in NPU-based LLM serving, on par with tensor parallelism and batch size. Each compiled bucket occupies its own HBM buffer, so memory bounds the total bucket count. Operators must therefore select a small set tailored to the workload.

The natural intuition is that smaller buckets should run faster: a smaller bucket pads less, so each request does less work. For ArXiv dataset, this suggests subdividing the 4096–8192 range at 512-token steps to push most requests into smaller, lighter buckets, rather than relying on the default {4096, 8192}. In practice, the finer-grained set increases E2E latency by 7% over the default.

The compiler optimizes only certain bucket shapes [17], and an optimized larger bucket can be faster than an unoptimized smaller one (Figure 9). Bucket latency on NPUs is therefore not monotonic in bucket size: it depends

on which shapes the compiler chose to optimized path. The optimal bucket set must account for this compiler behavior as well as the input/output length distribution of the workload, leaving operators with an empirical search across the full bucket space.

Each new bucket still requires its own profiling. LENS reduces this from a full deployment per bucket combination to two E2E measurements per bucket, turning the search from an exhaustive campaign into a quick exploration that surfaces the buckets the compiler actually optimizes.

Summary. Both case studies expose a common pattern: GPU-derived intuitions fail to predict NPU behavior, and the optimum can only be confirmed by direct measurement. The compiler and architecture differences that drive this also make analytical reasoning unreliable. LENS fills this gap as a practical search tool, letting operators explore the configuration space and identify the choices that yield the desired performance.

6. Future Work

While LENS achieves efficient profiling with only two measurements per bucket, its predictions remain confined to the NPU on which profiling was conducted. NeuSight [22] and Habitat [38] address the analogous limitation in the GPU domain by decomposing inference into kernel-level units and applying analytical scaling factors derived from hardware characteristics such as compute throughput and memory bandwidth. LENS, however, operates on bucket-level E2E latency without kernel decomposition, rendering kernel-level scaling factors directly inapplicable. Whether bucket-level E2E latency itself exhibits a consistent scaling relationship across NPUs, and what set of hardware features most accurately predicts such a relationship, remains an open question. If such a relationship can be established, LENS could be extended to support prediction across heterogeneous NPU platforms without requiring profiling on each. We leave the empirical characterization of cross-NPU scaling behavior to future work.

LENS currently predicts latency under an offline serving setting. Production serving environments introduce system-level factors that affect E2E latency, such as queuing delay, inter-stage communication overhead in pipeline parallelism, and request routing across heterogeneous accelerators. Extending LENS to system-level performance prediction that accounts for these factors is a direction for future work.

7. Related Work

DNN Performance Estimation. A substantial body of prior work has investigated DNN performance prediction [2, 22, 25, 31, 38]. Habitat [38] predicts kernel execution time on a target GPU by applying an analytical wave scaling factor derived from the roofline model, and integrates an ML-based approach to capture non-linearity in GPU computation. Vidur [2] profiles the operators arising in LLM inference by partitioning them into three categories (i.e., token-level operators, sequence-level operators, communication

operators) and predicts inference latency through random-forest-based interpolation. MaverIQ [25] minimizes profiling cost via a proxy model that loads only the unique layers of an LLM, and predicts inference latency across diverse configurations through an analytical communication model.

However, these studies are predicated on GPU architectures, and naive extensions that fail to account for the architectural characteristics of NPU systems (e.g., bucketing) yield substantial error rates. Furthermore, since each configuration on NPUs incurs a separate compilation, the granularity-adjustment approach of existing GPU methodologies cannot effectively reduce profiling overhead. By accounting for all of these characteristics, LENS attains low error rates with only two profiling measurements per bucket.

NPU Prior Work. Cycle-accurate simulators for NPU performance analysis have been developed in various forms, including multi-core NPU resource sharing (mNPUsim [16]), systolic array accelerators (SCALE-Sim v3 [32]), and PyTorch-integrated frameworks (PyTorchSim [36]).

However, all of these simulators share the common assumption that users must provide microarchitecture specifications as input, making them difficult to apply directly to commercial NPUs where microarchitectural details are not sufficiently disclosed. In addition, they do not explicitly address LLM inference characteristics such as the distinction between prefill and decode phases, or bucketing. LENS treats the microarchitecture and compiler as a black-box and predicts LLM inference latency using only bucket-level E2E profiling, complementing these simulation-based approaches.

8. Conclusion

This paper presents LENS, a tool that accurately predicts LLM inference latency on NPU systems. We characterize three challenges that preclude prior work from being applied to LLM inference latency prediction on commercial NPUs: the undisclosed microarchitecture, the unpredictability of compiler optimizations, and the bucketing-induced latency non-linearity. LENS addresses these challenges by measuring only two E2E execution times per bucket and composing them to predict E2E latency for arbitrary input-output combinations.

Across 248 cases on four commercial NPUs (Inferentia2, TPU v4, v5e, v6e), three models, and four datasets, LENS achieves a mean error of 2.15% and a median of 1.69%, with 89.9% of cases within 5% error. We further evaluate LENS against two methodologically related baselines, LLM Serving Sim 2.0 and MaverIQ, selected to validate our design choices in a controlled comparison. Both baselines exhibit substantial errors where their assumptions break (over 200% and over 70% respectively), while LENS maintains consistent accuracy with an average MAPE of 5.3%. Furthermore, two case studies demonstrate the need for LENS as a search tool for NPU serving operators facing configuration problems that defy GPU-derived intuitions.

References

- [1] “ShareGPT_Vicuna_unfiltered,” https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered, 2023, filtered and cleaned version of the ShareGPT dataset originally collected by RyokoAI.
- [2] A. Agrawal, N. Kedia, J. Mohan, A. Panwar, N. Kwatra, B. S. Gulavani, R. Ramjee, and A. Tumanov, “Vidur: A large-scale simulation framework for llm inference,” in *Proceedings of Machine Learning and Systems*, P. Gibbons, G. Pekhimenko, and C. D. Sa, Eds., vol. 6, 2024, pp. 351–366. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2024/file/b74a8de47d2b3c92836e0a011f48351-Paper-Conference.pdf
- [3] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, “Taming throughput-latency tradeoff in llm inference with sarathi-serve,” in *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI’24. USA: USENIX Association, 2024.
- [4] Amazon Web Services, “AWS Inferentia2 architecture,” <https://awsdocs-neuron.readthedocs-hosted.com/en/latest/general/arch/neuron-hardware/inferentia2.html>, 2024, accessed: 2026-05-15.
- [5] Anthropic, “Expanding our use of Google Cloud TPUs and services,” 2025, accessed: 2026-05-15. [Online]. Available: <https://www.anthropic.com/news/expanding-our-use-of-google-cloud-tpus-and-services>
- [6] —, “Powering the next generation of AI development with AWS,” 2025, accessed: 2026-05-15. [Online]. Available: <https://www.anthropic.com/news/anthropic-amazon-trainium>
- [7] J. Cho, H. Choi, G. Heo, and J. Park, “Lmservingsim 2.0: A unified simulator for heterogeneous and disaggregated llm serving infrastructure,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.23036>
- [8] A. Cohan, F. Dernoncourt, D. S. Kim, T. Bui, S. Kim, W. Chang, and N. Goharian, “A discourse-aware attention model for abstractive summarization of long documents,” in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, M. Walker, H. Ji, and A. Stent, Eds. New Orleans, Louisiana: Association for Computational Linguistics, Jun. 2018, pp. 615–621. [Online]. Available: <https://aclanthology.org/N18-2097/>
- [9] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “Flashattention: fast and memory-efficient exact attention with io-awareness,” in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, ser. NIPS ’22. Red Hook, NY, USA: Curran Associates Inc., 2022.
- [10] A. Fan, M. Lewis, and Y. Dauphin, “Hierarchical neural story generation,” in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, I. Gurevych and Y. Miyao, Eds. Melbourne, Australia: Association for Computational Linguistics, Jul. 2018, pp. 889–898. [Online]. Available: <https://aclanthology.org/P18-1082/>
- [11] P. Garcia, F. Agullo, Y. Zhu, C. Wang, E. Lee, O. Tardieu, J. Torres, and J. Berral, “Mind the memory gap: Unveiling gpu bottlenecks in large-batch llm inference,” 07 2025, pp. 277–287.
- [12] Google Cloud, “Cloud TPU v5e system architecture,” <https://cloud.google.com/tpu/docs/v5e>, 2024, accessed: 2026-05-15.
- [13] —, “Cloud TPU v6e (trillium) system architecture,” <https://cloud.google.com/tpu/docs/v6e>, 2024, accessed: 2026-05-15.
- [14] —, “Trillium TPU is GA,” 2024, accessed: 2026-05-15. [Online]. Available: <https://cloud.google.com/blog/products/compute/trillium-tpu-is-ga>
- [15] H. Ham, W. Yang, Y. Shin, O. Woo, G. Heo, S. Lee, J. Park, and G. Kim, “Onnxim: A fast, cycle-level multi-core npu simulator,” *IEEE Comput. Archit. Lett.*, vol. 23, no. 2, p. 219–222, Jul. 2024. [Online]. Available: <https://doi.org/10.1109/LCA.2024.3484648>
- [16] S. Hwang, S. Lee, J. Kim, H. Kim, and J. Huh, “mNPUsim: Evaluating the effect of sharing resources in multi-core NPUs,” in *2023 IEEE International Symposium on Workload Characterization (IISWC)*, 2023, pp. 167–179. [Online]. Available: <https://doi.org/10.1109/IISWC59245.2023.00018>
- [17] J. Jiang, Y. Chen, B. A. Hechtman, F. Zhang, and Y. Mu, “Ragged paged attention: A high-performance and flexible llm inference kernel for tpu,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.15464>
- [18] N. P. Jouppi, D. Hyun Yoon, M. Ashcraft, M. Gottscho, T. B. Jablin, G. Kurian, J. Laudon, S. Li, P. Ma, X. Ma, T. Norrie, N. Patil, S. Prasad, C. Young, Z. Zhou, and D. Patterson, “Ten lessons from three generations shaped google’s tpuv4i : Industrial product,” in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 1–14.
- [19] N. P. Jouppi, G. Kurian, S. Li, P. Ma, R. Nagarajan, L. Nai, N. Patil, S. Subramanian, A. Swing, B. Towles, C. Young, X. Zhou, Z. Zhou, and D. Patterson, “TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA)*, 2023. [Online]. Available: <https://doi.org/10.1145/3579371.3589350>
- [20] N. P. Jouppi, C. Young, N. Patil, D. Patterson *et al.*, “In-datacenter performance analysis of a tensor processing unit,” *SIGARCH Comput. Archit. News*, vol. 45, no. 2, p. 1–12, Jun. 2017. [Online]. Available: <https://doi.org/10.1145/3140659.3080246>
- [21] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” in *Proceedings of the 29th Symposium on Operating Systems Principles*, ser. SOSP ’23. New York, NY, USA: Association for Computing Machinery, 2023, p. 611–626. [Online]. Available: <https://doi.org/10.1145/3600006.3613165>
- [22] S. Lee, A. Phanishayee, and D. Mahajan, “Forecasting gpu performance for deep learning training and inference,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. ACM, Mar. 2025, pp. 493–508. [Online]. Available: <http://dx.doi.org/10.1145/3669940.3707265>
- [23] J. Lew, D. A. Shah, S. Pati, S. Cattell, M. Zhang, A. Sandhupatla, C. Ng, N. Goli, M. D. Sinclair, T. G. Rogers, and T. M. Aamodt, “Analyzing machine learning workloads using a detailed gpu simulator,” in *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2019, pp. 151–152.
- [24] Y. Li, Y. Sun, and A. Jog, “Path forward beyond simulators: Fast and accurate gpu execution time prediction for dnn workloads,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO ’23. New York, NY, USA: Association for Computing Machinery, 2023, p. 380–394. [Online]. Available: <https://doi.org/10.1145/3613424.3614277>
- [25] D. Liakopoulos, P. Sinha, T. Hu, M. Lee, and N. J. Yadwadkar, “Maveriq: Fingerprint-guided extrapolation and fragmentation-aware layering for intent-based llm serving,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1676–1696. [Online]. Available: <https://doi.org/10.1145/3712285.3759867>
- [26] Llama Team, AI @ Meta, “The llama 3 herd of models,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [27] R. Nallapati, B. Zhou, C. dos Santos, Ç. Gülçehre, and B. Xiang, “Abstractive text summarization using sequence-to-sequence RNNs and beyond,” in *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning*, S. Riezler and Y. Goldberg, Eds. Berlin, Germany: Association for Computational Linguistics, Aug. 2016, pp. 280–290. [Online]. Available: <https://aclanthology.org/K16-1028/>

- [28] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia, "Efficient large-scale language model training on gpu clusters using megatron-lm," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '21. New York, NY, USA: Association for Computing Machinery, 2021. [Online]. Available: <https://doi.org/10.1145/3458817.3476209>
- [29] A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer, "Timeloop: A systematic approach to dnn accelerator evaluation," in *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2019, pp. 304–315.
- [30] P. Patel, E. Choukse, C. Zhang, A. Shah, I. n. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative llm inference using phase splitting," in *Proceedings of the 51st Annual International Symposium on Computer Architecture*, ser. ISCA '24. IEEE Press, 2025, p. 118–132. [Online]. Available: <https://doi.org/10.1109/ISCA59077.2024.00019>
- [31] R. Patwari, A. Sirasao, and D. Das, "Forecasting llm inference performance via hardware-agnostic analytical modeling," 2025. [Online]. Available: <https://arxiv.org/abs/2508.00904>
- [32] R. Raj, S. Banerjee, N. Chandra, Z. Wan, J. Tong, A. Samajdhar, and T. Krishna, "Scale-sim v3: a modular cycle-accurate systolic accelerator simulator for end-to-end system analysis," in *2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2025, pp. 186–200.
- [33] The JAX Authors, "Just-in-time compilation," <https://docs.jax.dev/en/latest/jit-compilation.html>, accessed: 2026-05-19.
- [34] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. USA: Curran Associates, Inc., 2017. [Online]. Available: <https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf>
- [35] D. Xu, H. Zhang, L. Yang, R. Liu, G. Huang, M. Xu, and X. Liu, "Fast on-device llm inference with npus," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 445–462. [Online]. Available: <https://doi.org/10.1145/3669940.3707239>
- [36] W. Yang, Y. Shin, O. Woo, G. Park, H. Ham, J. Kang, J. Park, and G. Kim, "Pytorchsim: A comprehensive, fast, and accurate npu simulation framework," in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1363–1380. [Online]. Available: <https://doi.org/10.1145/3725843.3756045>
- [37] W. Yin, D. Xu, M. Xu, G. Huang, and X. Liu, "Shadownpu: System and algorithm co-design for npu-centric on-device llm inference," 2026. [Online]. Available: <https://arxiv.org/abs/2508.16703>
- [38] G. X. Yu, Y. Gao, P. Golikov, and G. Pekhimenko, "Habitat: A Runtime-Based computational performance predictor for deep neural network training," in *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, Jul. 2021, pp. 503–521. [Online]. Available: <https://www.usenix.org/conference/atc21/presentation/you>
- [39] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for Transformer-Based generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 521–538. [Online]. Available: <https://www.usenix.org/conference/osdi22/presentation/you>
- [40] Y. Zhao, E. Johnson, P. Chatarasi, V. S. Adve, and S. Misailovic, "Neptune: Advanced ML operator fusion for locality and parallelism on GPUs," *Proceedings of the ACM on Programming Languages*, vol. 10, no. PLDI, 2026.
- [41] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 193–210. [Online]. Available: <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>
- [42] H. Zhu, A. Phanishayee, and G. Pekhimenko, "Daydream: Accurately estimating the efficacy of optimizations for DNN training," in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, Jul. 2020, pp. 337–352. [Online]. Available: <https://www.usenix.org/conference/atc20/presentation/zhu-hongyu>